Лабораторная работа №2. Задача кластеризации

Студент: Васютинский Вадим Александрович **Вариант:** 8 (`lab2-08.xlsx`)

Цель: научиться применять модели кластеризации библиотеки scikit-learn к набору эмпирических данных.

Результат: число кластеров и обученная модель.

In [1]:
```
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

from sklearn.model_selection import train_test_split
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.metrics import (
    silhouette_score,
    davies_bouldin_score,
    calinski_harabasz_score,
    adjusted_rand_score,
)

RANDOM_STATE = 42
DATA_PATH = Path('..') / '..' / 'tasks' / 'lab2' / 'dataset' / 'lab2-08.x
```

1. Загрузка данных

In [2]:
```
data = pd.read_excel(DATA_PATH)
data.columns = ['x1', 'x2']
data.head()
```

Out[2]:

	x1	x2
0	3.757193	-1.042462
1	1.788737	-1.772939
2	-2.608214	-1.925894
3	-1.871252	-0.173571
4	-3.728688	0.179285

In [3]:
```
data.info()
data.describe()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   x1      1000 non-null   float64
 1   x2      1000 non-null   float64
dtypes: float64(2)
memory usage: 15.8 KB
```

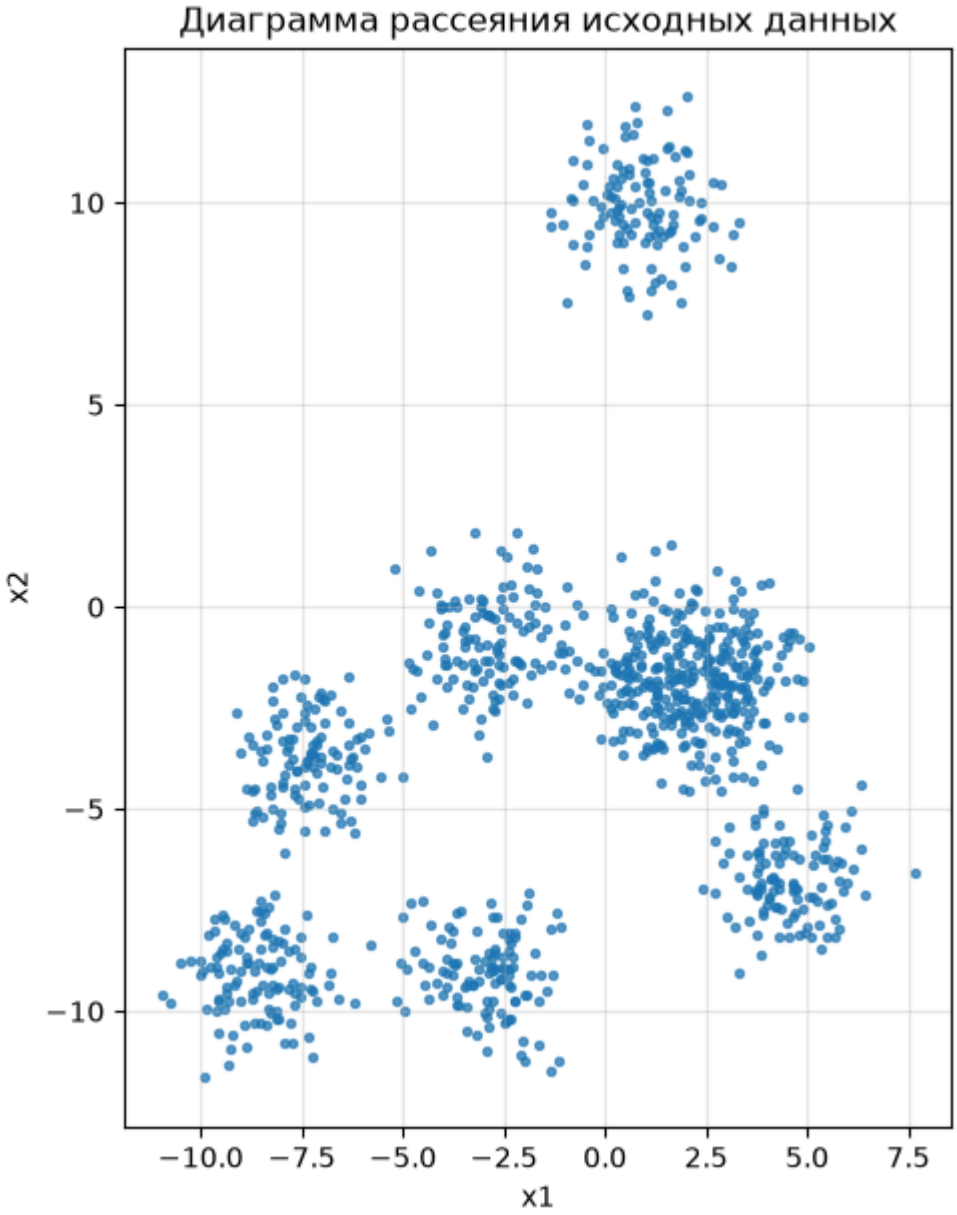
Out[3]:

	x1	x2
count	1000.000000	1000.000000
mean	-1.141497	-2.756044
std	4.446475	5.513910
min	-10.954978	-11.648055
25%	-4.067796	-7.252962
50%	0.250433	-2.627119
75%	2.484677	-0.963996
max	7.643615	12.613797

Файл содержит 1000 объектов с двумя числовыми признаками, пропусков нет. Оба признака измерены в одном масштабе (примерно от –12 до 13), поэтому нормализация не нужна.

2. Визуализация исходных данных

In [4]:
```
fig, ax = plt.subplots(figsize=(7, 7))
ax.scatter(data['x1'], data['x2'], s=8, alpha=0.7)
ax.set_xlabel('x1')
ax.set_ylabel('x2')
ax.set_title('Диаграмма рассеяния исходных данных')
ax.set_aspect('equal')
ax.grid(alpha=0.3)
plt.show()
```



На диаграмме видны компактные группы примерно круглой формы и близкого размера. Визуально групп 7; две из них (в центре, около точек (−3, −1) и (2, −2)) расположены близко и частично соприкасаются.

3. Оценка числа кластеров

Сначала разделим данные на обучающую (80 %) и тестовую (20 %) выборки. Число кластеров и модель выбираются только по обучающей выборке, тестовая выборка нужна для проверки в п. 6.

In [5]:
```
X = data[['x1', 'x2']].to_numpy()
X_train, X_test = train_test_split(X, test_size=0.2, random_state=RANDOM_
print(f'Обучающая выборка: {len(X_train)}, тестовая: {len(X_test)}')
```

```
Обучающая выборка: 800, тестовая: 200
```

Для $k = 2, \ldots, 10$ обучим KMeans и посчитаем:

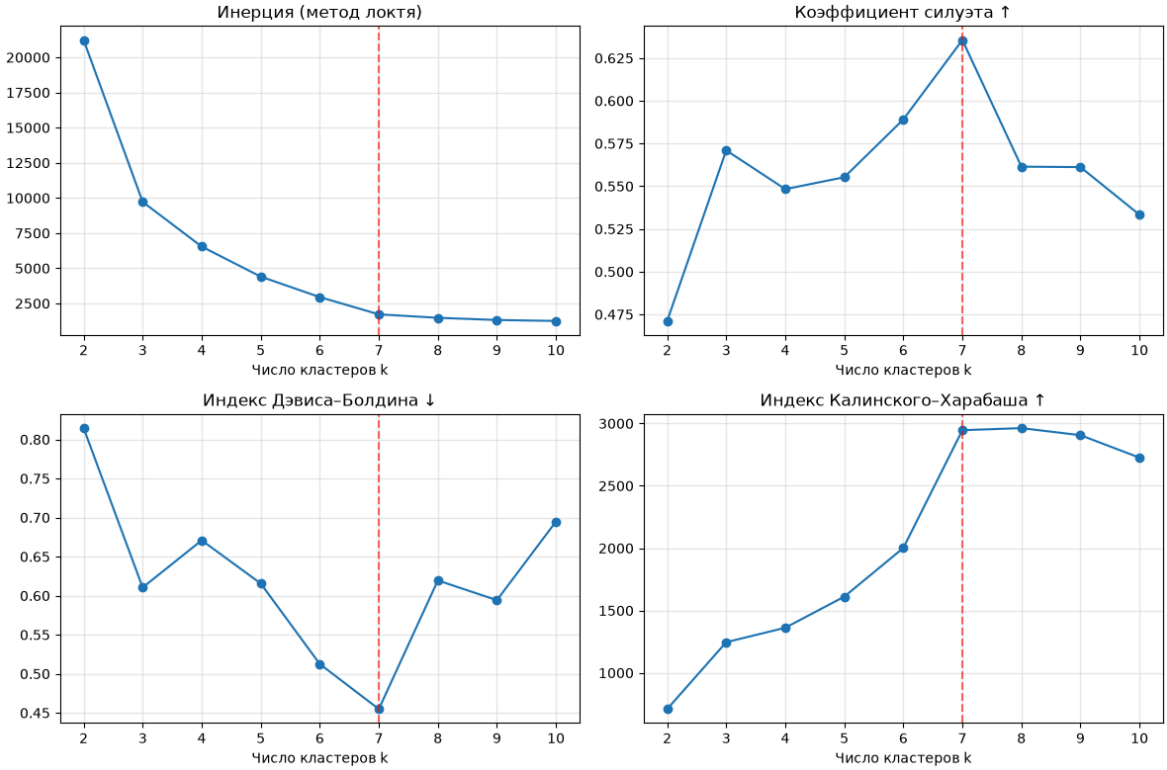
- **инерцию** (сумму квадратов расстояний до центров), чтобы применить метод «локтя»;
- **коэффициент силуэта** (чем больше, тем лучше, максимум 1);
- **индекс Дэвиса–Болдина** (чем меньше, тем лучше);
- **индекс Калинского–Харабаша** (чем больше, тем лучше).

In [6]:
```
k_range = range(2, 11)
rows = []
for k in k_range:
    model = KMeans(n_clusters=k, n_init=10, random_state=RANDOM_STATE).fi
    rows.append({
        'k': k,
        'inertia': model.inertia_,
        'silhouette': silhouette_score(X_train, model.labels_),
        'davies_bouldin': davies_bouldin_score(X_train, model.labels_),
        'calinski_harabasz': calinski_harabasz_score(X_train, model.label
    })
k_scores = pd.DataFrame(rows).set_index('k')
k_scores.round(3)
```

Out[6]:

k	inertia	silhouette	davies_bouldin	calinski_harabasz
2	21231.075	0.471	0.814	709.642
3	9720.071	0.571	0.610	1245.951
4	6541.056	0.548	0.671	1361.702
5	4404.591	0.555	0.616	1611.151
6	2948.389	0.589	0.513	2001.524
7	1722.350	0.636	0.455	2945.930
8	1475.499	0.561	0.619	2962.539
9	1320.204	0.561	0.594	2905.845
10	1251.626	0.533	0.695	2725.812

In [7]:
```
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
titles = {
    'inertia': 'Инерция (метод локтя)',
    'silhouette': 'Коэффициент силуэта ↑',
    'davies_bouldin': 'Индекс Дэвиса–Болдина ↓',
    'calinski_harabasz': 'Индекс Калинского–Харабаша ↑',
}
for ax, (column, title) in zip(axes.ravel(), titles.items()):
    ax.plot(k_scores.index, k_scores[column], marker='o')
    ax.axvline(7, color='red', linestyle='--', alpha=0.6)
    ax.set_title(title)
    ax.set_xlabel('Число кластеров k')
    ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



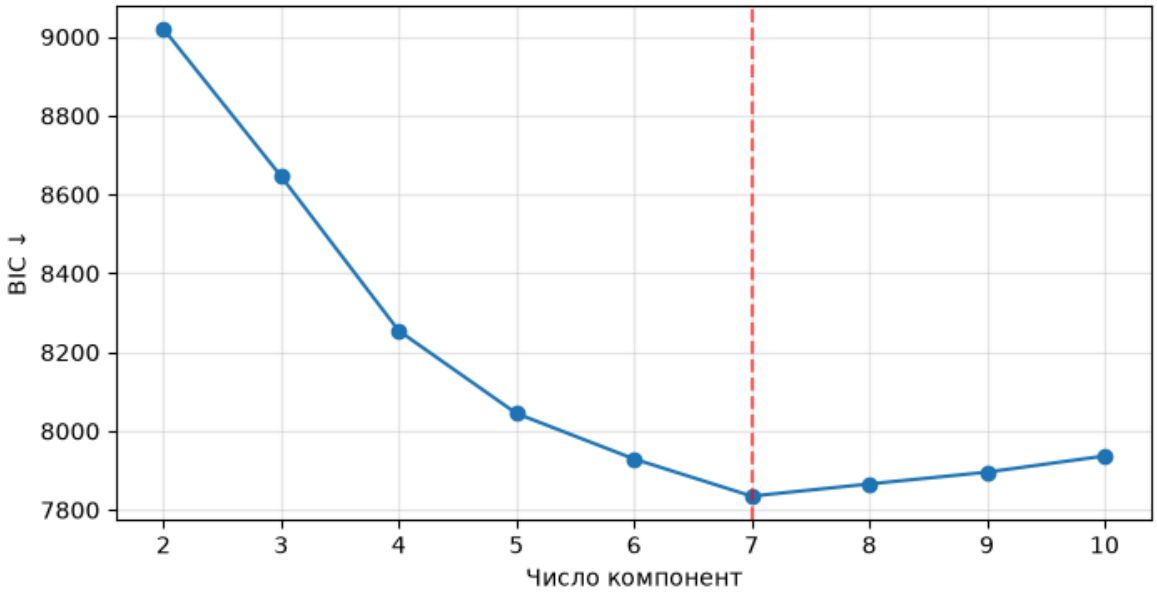
In [8]:
```
print('k по максимуму силуэта:              ', k_scores['silhouette'].idxma
print('k по минимуму Дэвиса–Болдина:     ', k_scores['davies_bouldin'].id
print('k по максимуму Калинского–Харабаша:', k_scores['calinski_harabasz'
```

```
k по максимуму силуэта:              7
k по минимуму Дэвиса–Болдина:      7
k по максимуму Калинского–Харабаша: 8
```

Для сравнения посчитаем информационный критерий BIC для гауссовой смеси: он оценивает, сколько нормальных компонент лучше всего описывают данные с учётом штрафа за число параметров.

In [9]:
```
bic = pd.Series(
    {k: GaussianMixture(n_components=k, n_init=5, random_state=RANDOM_STA
     for k in k_range},
    name='BIC',
)

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(bic.index, bic.values, marker='o')
ax.axvline(7, color='red', linestyle='--', alpha=0.6)
ax.set_xlabel('Число компонент')
ax.set_ylabel('BIC ↓')
ax.grid(alpha=0.3)
plt.show()
print('k по минимуму BIC:', bic.idxmin())
```




```
k по минимуму BIC: 7
```

На графике инерции после $k = 7$ виден «локоть», а коэффициент силуэта, индекс Дэвиса–Болдина и BIC достигают оптимума именно при $k = 7$. Индекс Калинского–Харабаша формально максимален при $k = 8$, но отличается от значения при $k = 7$ всего на ~0.5 %, то есть фактически выходит на плато начиная с $k = 7$. При $k = 8$ силуэт заметно падает (0.64 → 0.56): восьмой кластер получается разрезанием одной из групп пополам. Вместе с визуальной оценкой это даёт **число кластеров: 7.**

4. Выбор модели кластеризации

Кластеры компактные, выпуклые, примерно круглые и с близким разбросом, шум и выбросы почти отсутствуют. Для таких данных хорошо подходит **KMeans**: он ищет k центров, минимизируя внутрикластерный разброс, и умеет относить новые точки к ближайшему центру (метод `predict`), что нужно для проверки на тестовой выборке.

В качестве альтернативы рассмотрим **гауссову смесь** (`GaussianMixture`): она допускает эллиптические кластеры разного размера и тоже умеет предсказывать кластер для новых точек. Методы на основе плотности (DBSCAN) здесь менее удобны: две центральные группы соприкасаются, и DBSCAN легко сольёт их в одну, к тому же он не умеет предсказывать кластер для новых точек.

5. Обучение модели на обучающей выборке

In [10]:
```
N_CLUSTERS = 7

kmeans = KMeans(n_clusters=N_CLUSTERS, n_init=10, random_state=RANDOM_STA
gmm = GaussianMixture(n_components=N_CLUSTERS, n_init=5, random_state=RAN

centers = pd.DataFrame(kmeans.cluster_centers_, columns=['x1', 'x2'])
```

```
centers['size'] = np.bincount(kmeans.labels_)
centers.round(3)
```

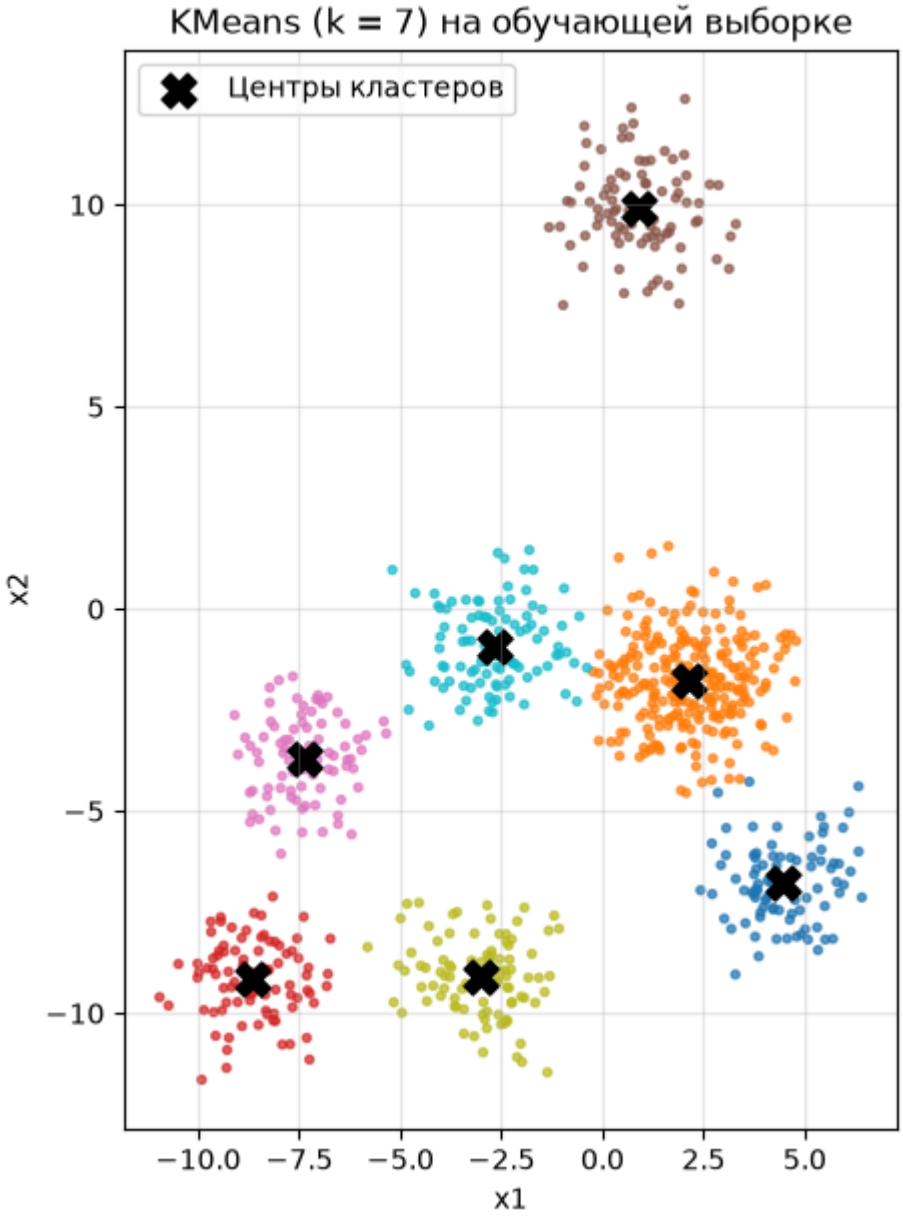
Out[10]:

	x1	x2	size
0	4.466	-6.756	91
1	2.134	-1.777	251
2	-8.643	-9.153	86
3	0.909	9.889	88
4	-7.388	-3.702	88
5	-3.030	-9.081	97
6	-2.686	-0.920	99

Шесть кластеров содержат примерно по 90–100 точек, а центральный кластер около (2.1, –1.8) примерно в 2.5 раза больше. При этом его разброс лишь немного превышает разброс остальных групп, то есть это одна более плотная группа, а не две слившиеся: при $k = 8$ ни KMeans, ни гауссова смесь не выделяют внутри неё отдельной структуры. Возможно, в данных здесь совпадают несколько источников, но по геометрии точек различить их нельзя.

In [11]:

```
fig, ax = plt.subplots(figsize=(7, 7))
ax.scatter(X_train[:, 0], X_train[:, 1], c=kmeans.labels_, cmap='tab10',
ax.scatter(*kmeans.cluster_centers_.T, c='black', marker='X', s=150, labe
ax.set_xlabel('x1')
ax.set_ylabel('x2')
ax.set_title('KMeans (k = 7) на обучающей выборке')
ax.set_aspect('equal')
ax.legend()
ax.grid(alpha=0.3)
plt.show()
```



6. Проверка качества на тестовой выборке

Истинных меток кластеров в данных нет, поэтому качество проверяем двумя способами:

1. **Внутренние метрики на тесте.** Предсказываем кластеры для тестовых точек обученной моделью и считаем силуэт и индексы. Если значения близки к значениям на обучающей выборке, разбиение переносится на новые данные.
2. **Устойчивость.** Обучаем такую же модель на тестовой выборке независимо и сравниваем её разбиение тестовых точек с предсказанием модели, обученной на обучающей выборке, по скорректированному индексу Рэнда (ARI). ARI = 1 означает полное совпадение разбиений с точностью до перенумерации кластеров.

In [12]:
```
def cluster_metrics(X_part, labels):
    return {
```

```
        'silhouette': silhouette_score(X_part, labels),
        'davies_bouldin': davies_bouldin_score(X_part, labels),
        'calinski_harabasz': calinski_harabasz_score(X_part, labels),
    }


models = {'KMeans': kmeans, 'GaussianMixture': gmm}
rows = []
for name, model in models.items():
    test_labels = model.predict(X_test)
    refit_labels = type(model)(**model.get_params()).fit(X_test).predict(
    for split, X_part, labels in [('train', X_train, model.predict(X_trai
        rows.append({'model': name, 'split': split, **cluster_metrics(X_p
    rows[-1]['ARI (refit on test)'] = adjusted_rand_score(test_labels, re

quality = pd.DataFrame(rows).set_index(['model', 'split'])
quality.round(3)
```

Out[12]:

model	split	silhouette	davies_bouldin	calinski_harabasz	ARI (refit on test)
KMeans	train	0.636	0.455	2945.930	NaN
	test	0.604	0.476	638.807	0.97
GaussianMixture	train	0.636	0.452	2924.486	NaN
	test	0.606	0.470	639.717	0.97

In [13]:
```
agreement = adjusted_rand_score(kmeans.predict(X), gmm.predict(X))
print(f'Согласованность KMeans и GaussianMixture на всех данных (ARI): {a
```

```
Согласованность KMeans и GaussianMixture на всех данных (ARI): 0.979
```

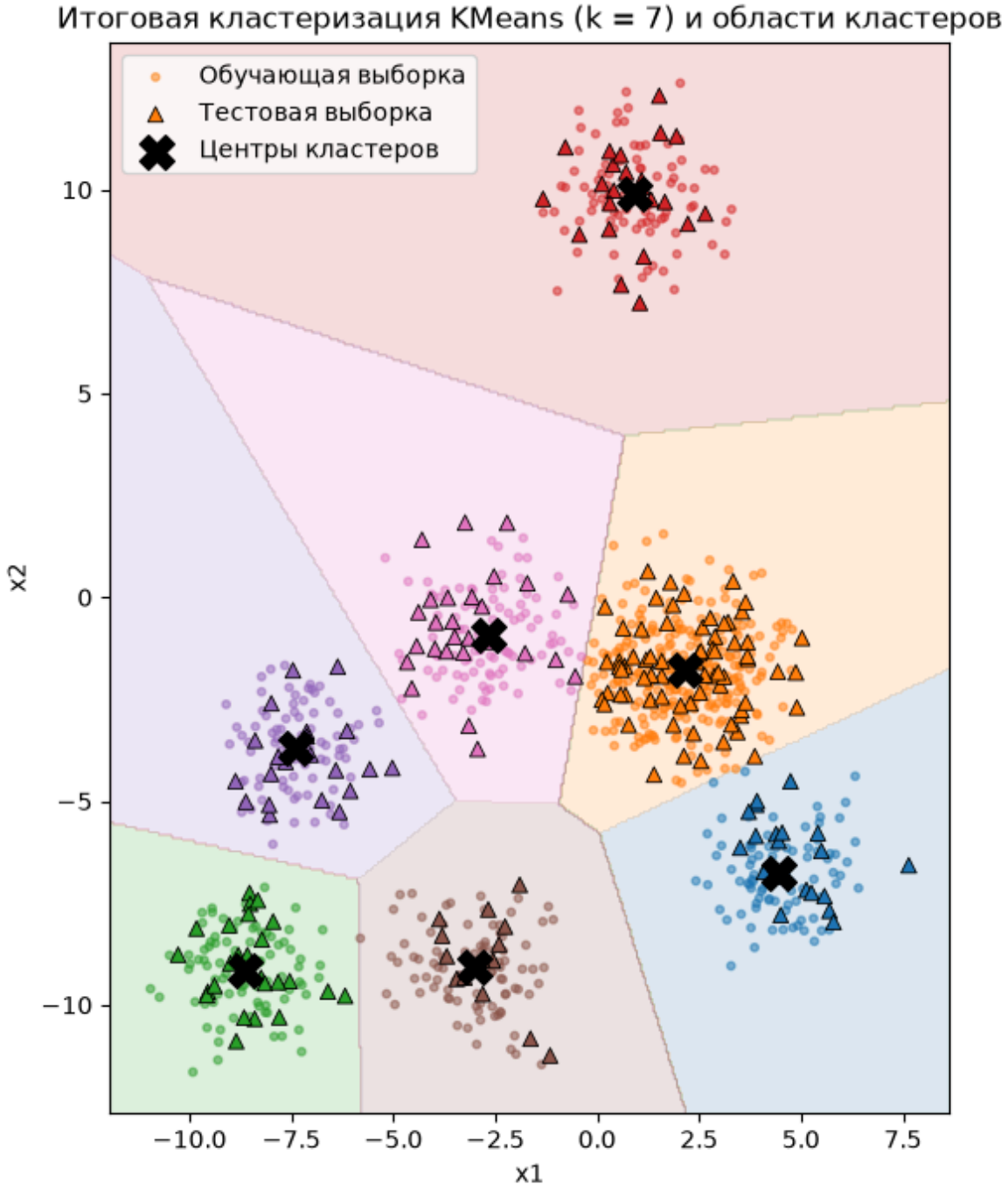
Силуэт и индекс Дэвиса–Болдина на тестовой выборке близки к значениям на обучающей (индекс Калинского–Харабаша растёт с числом объектов, поэтому его значения на выборках разного размера напрямую не сравниваются), а модель, обученная заново на одних только тестовых точках, даёт практически то же разбиение (ARI = 0.97). KMeans и гауссова смесь почти полностью согласуются друг с другом, так что найденная структура из семи кластеров устойчива и не зависит от выбора модели.

7. Визуализация результата

In [14]:
```
grid_x1, grid_x2 = np.meshgrid(
    np.linspace(X[:, 0].min() - 1, X[:, 0].max() + 1, 400),
    np.linspace(X[:, 1].min() - 1, X[:, 1].max() + 1, 400),
)
regions = kmeans.predict(np.c_[grid_x1.ravel(), grid_x2.ravel()]).reshape
cluster_cmap = ListedColormap(plt.cm.tab10.colors[:N_CLUSTERS])

fig, ax = plt.subplots(figsize=(8, 8))
ax.contourf(grid_x1, grid_x2, regions, levels=np.arange(N_CLUSTERS + 1) -
ax.scatter(X_train[:, 0], X_train[:, 1], c=kmeans.labels_, cmap=cluster_c
           s=10, alpha=0.5, label='Обучающая выборка')
```

```
ax.scatter(X_test[:, 0], X_test[:, 1], c=kmeans.predict(X_test), cmap=clu
           s=40, marker='^', edgecolors='black', linewidths=0.6, label='T
ax.scatter(*kmeans.cluster_centers_.T, c='black', marker='X', s=200, labe
ax.set_xlabel('x1')
ax.set_ylabel('x2')
ax.set_title('Итоговая кластеризация KMeans (k = 7) и области кластеров')
ax.set_aspect('equal')
ax.legend(loc='upper left')
plt.show()
```

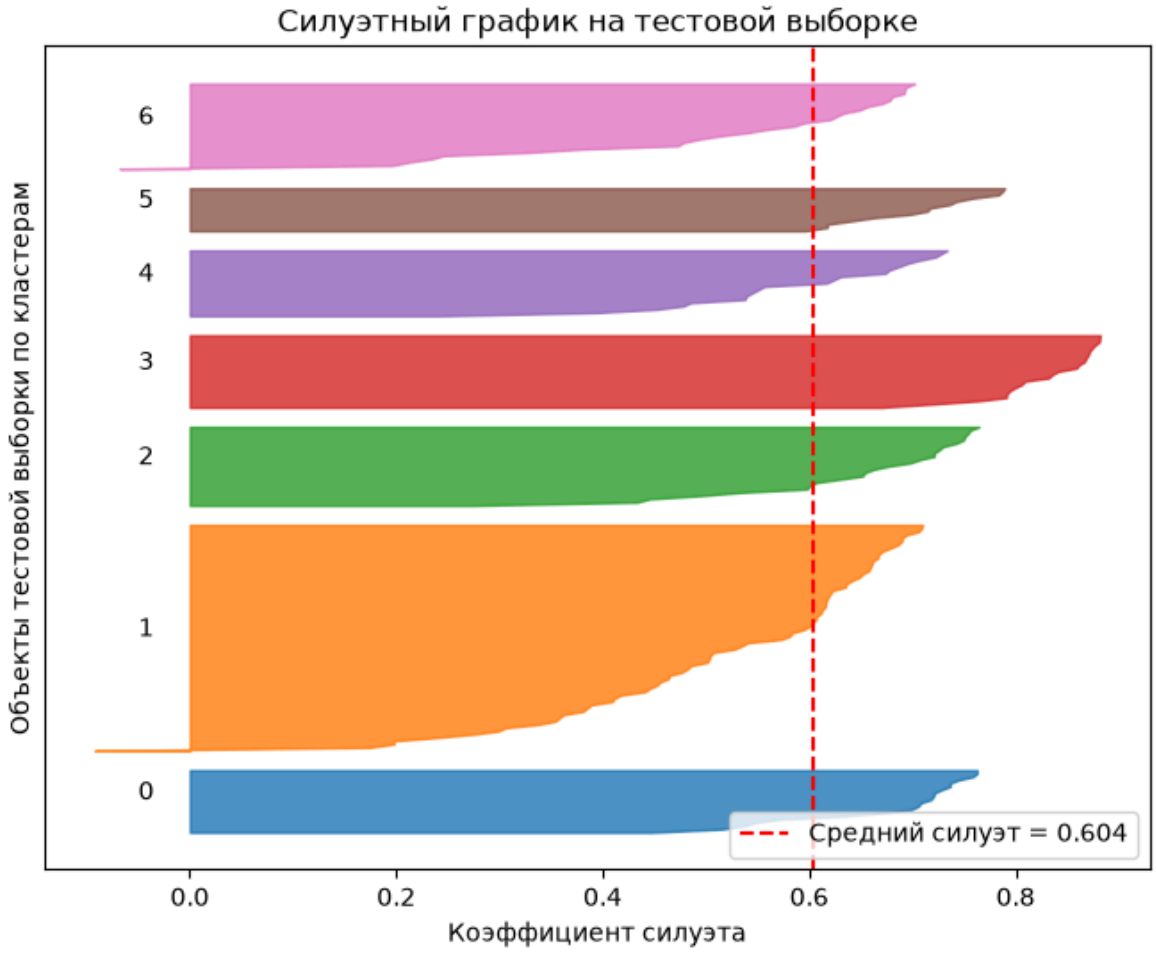


In [15]:
```
from sklearn.metrics import silhouette_samples

test_labels = kmeans.predict(X_test)
sil = silhouette_samples(X_test, test_labels)

fig, ax = plt.subplots(figsize=(8, 6))
y_lower = 0
for cluster in range(N_CLUSTERS):
```

```
    values = np.sort(sil[test_labels == cluster])
    ax.fill_betweenx(np.arange(y_lower, y_lower + len(values)), 0, values
                     color=plt.cm.tab10(cluster), alpha=0.8)
    ax.text(-0.05, y_lower + len(values) / 2, str(cluster))
    y_lower += len(values) + 5
ax.axvline(sil.mean(), color='red', linestyle='--', label=f'Средний силуэ
ax.set_xlabel('Коэффициент силуэта')
ax.set_ylabel('Объекты тестовой выборки по кластерам')
ax.set_yticks([])
ax.set_title('Силуэтный график на тестовой выборке')
ax.legend(loc='lower right')
plt.show()
```



Почти у всех объектов силуэт положительный. Более низкие значения встречаются в двух соприкасающихся центральных кластерах, у точек на их общей границе, что ожидаемо.

Вывод

1. Данные варианта 8 состоят из 1000 точек на плоскости, которые образуют компактные группы примерно круглой формы.
2. Число кластеров оценено методом локтя, коэффициентом силуэта, индексами Дэвиса–Болдина и Калинского–Харабаша и критерием BIC для гауссовой смеси. Силуэт, индекс Дэвиса–Болдина, BIC и визуальный анализ указывают на **7 кластеров**; индекс Калинского–Харабаша при $k = 7$ и $k = 8$ практически одинаков.

3. В качестве модели выбран **KMeans** с $k = 7$, обученный на 80 % данных. Центры кластеров приведены в п. 5.
4. На тестовой выборке силуэт (0.60) и индекс Дэвиса–Болдина (0.48) близки к значениям на обучающей (0.64 и 0.46), а повторное обучение на тестовых данных воспроизводит то же разбиение. Гауссова смесь даёт практически такое же разбиение, как KMeans. Значит, модель устойчива и корректно кластеризует новые данные.